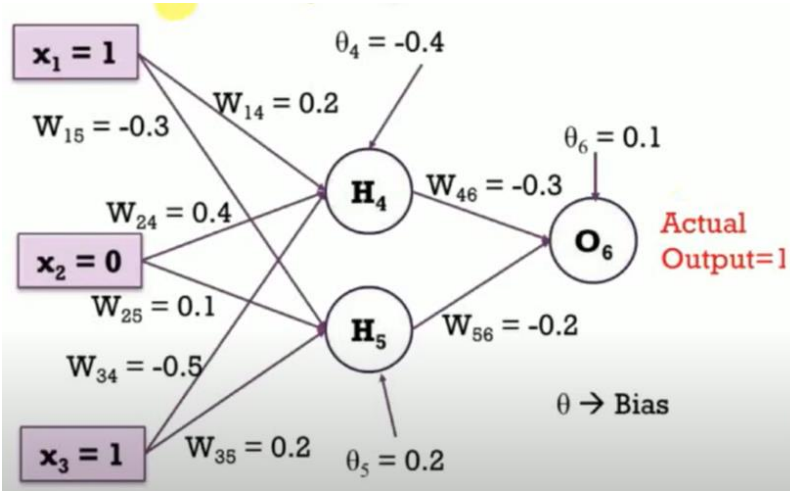
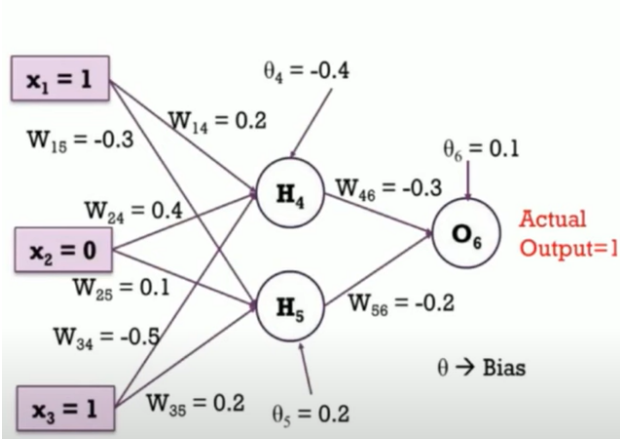
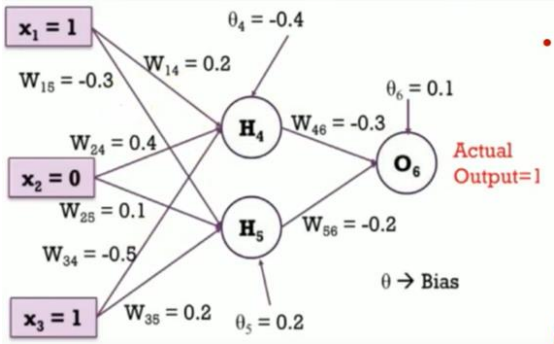


Multilayer Perceptron Network

Machine Learning



Assume that the neurons have a sigmoid activation function, perform a forward pass and a backward pass on the network. Assume that the actual output of y is 1 and learning rate is 0.9. Perform another forward pass.



Error $= y_{target} - y_6 = 0.526$

- Forward Pass: Compute output for y4, y5 and y6.

$$a_j = \sum_j (w_{i,j} * x_i) \qquad yj = F(aj) = \frac{1}{1+e^{-a_j}}$$

$a_4 = (w_{14} * x_1) + (w_{24} * x_2) + (w_{34} * x_3) + \theta_4$
$= (0.2 * 1) + (0.4 * 0) + (-0.5 * 1) + (-0.4) = -0.7$

$O(H_4) = y_4 = f(a_4) = 1/(1 + e^{0.7}) = 0.332$

$a_5 = (w_{15} * x_1) + (w_{25} * x_2) + (w_{35} * x_3) + \theta_5$
$= (-0.3 * 1) + (0.1 * 0) + (0.2 * 1) + (0.2) = 0.1$

$O(H_5) = y_5 = f(a_5) = 1/(1 + e^{-0.1}) = 0.525$

$a_6 = (w_{46} * H_4) + (w_{56} * H_5) + \theta_6$
$= (-0.3 * 0.332) + (-0.2 * 0.525) + 0.1 = -0.105$

$O(O_6) = y_6 = f(a_6) = 1/(1 + e^{0.105}) = \mathbf{0.474}$

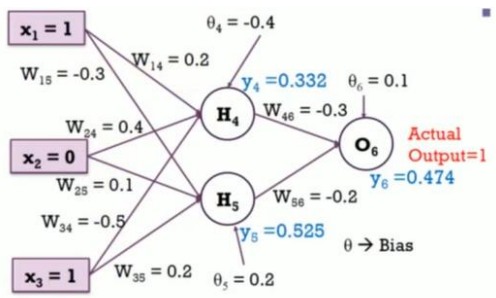
- Each weight changed by:

$$\Delta w_{ji} = \eta \delta_j o_i$$

$$\delta_j = o_j(1-o_j)(t_j - o_j) \qquad \text{if } j \text{ is an output unit}$$

$$\delta_j = o_j(1-o_j)\sum_k \delta_k w_{kj} \qquad \text{if } j \text{ is a hidden unit}$$

- where η is a constant called the learning rate
- tj is the correct teacher output for unit j
- δj is the error measure for unit j



- Backward Pass: Compute δ4, δ5 and δ6.

For output unit:

$\delta_6 = y_6(1-y_6)\,(y_{target} - y_6)$

$= 0.474*(1-0.474)*(1-0.474) = 0.1311$

For hidden unit:

$\delta_5 = y_5(1-y_5)\, w_{56} * \delta_6$

$= 0.525*(1 - 0.525)*(-0.2 * 0.1311) = -0.0065$

$= y_4(1-y_4)\, w_{46} * \delta_6$

$= 0.332*(1 - 0.332)* (-0.3 * 0.1331) = -0.0087$

Compute new weights

$$\Delta w_{ji} = \eta \delta_j o_i$$

$\Delta w_{46} = \eta\, \delta_6\, y_4 = 0.9 * 0.1311 * 0.332 = 0.03917$

w_{46} (new) = $\Delta w_{46} + w_{46}$(old) = 0.03917 + (-0.3) = -0.261

$\Delta w_{14} = \eta\, \delta_4\, x_1 = 0.9 * -0.0087 * 1 = -0.0078$

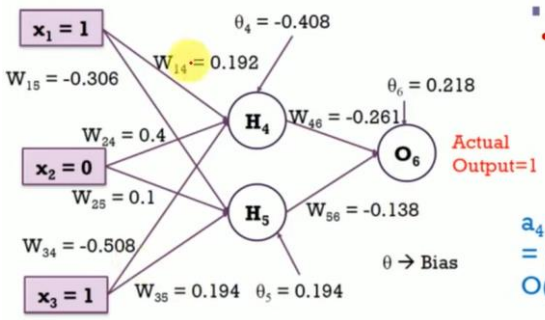
w_{14} (new) = $\Delta w_{14} + w_{14}$(old) = -0.0078 + 0.2 = 0.192

- Similarly, update all other weights

i	j	w_{ij}	δ_j	x_i	η	Updated w_{ij}
4	6	-0.3	0.1311	0.332	0.9	-0.261
5	6	-0.2	0.1311	0.525	0.9	-0.138
1	4	0.2	-0.0087	1	0.9	0.192
1	5	-0.3	-0.0065	1	0.9	-0.306
2	4	0.4	-0.0087	0	0.9	0.4
2	5	0.1	-0.0065	0	0.9	0.1
3	4	-0.5	-0.0087	1	0.9	-0.508
3	5	0.2	-0.0065	1	0.9	0.194

Similarly, update bais weights

θ_j	Previous θ_j	δ_j	η	Updated θ_j
Θ_6	0.1	0.1311	0.9	0.218
Θ_5	0.2	-0.0065	0.9	0.194
Θ_4	-0.4	-0.0087	0.9	-0.408



Error $= y_{target} - y_6 = 0.485$

- Forward Pass: Compute output for y4, y5 and y6.

$$a_j = \sum_j (w_{i,j} * x_i) \qquad yj = F(aj) = \frac{1}{1+e^{-a_j}}$$

$a_4 = (w_{14} * x_1) + (w_{24} * x_2) + (w_{34} * x_3) + \theta_4$
$= (0.192 * 1) + (0.4 * 0) + (-0.508 * 1) + (-0.408) = -0.724$
$O(H_4) = y_4 = f(a_4) = 1/(1 + e^{0.724}) = 0.327$

$a_5 = (w_{15} * x_1) + (w_{25} * x_2) + (w_{35} * x_3) + \theta_5$
$= (-0.306 * 1) + (0.1 * 0) + (0.194 * 1) + (0.194) = 0.082$
$O(H_5) = y_5 = f(a_5) = 1/(1 + e^{-0.082}) = 0.520$

$a_6 = (w_{46} * H_4) + (w_{56} * H_5) + \theta_6$
$= (-0.261 * 0.327) + (-0.138 * 0.520) + 0.218 = 0.061$
$O(O_6) = y_6 = f(a_6) = 1/(1 + e^{-0.061}) = \mathbf{0.515}$ (Network Output)

Task

Utilize the Pandas library and apply matrix multiplication for weights, input layer values, and hidden layer values. Implement the neural network code in Python from scratch.